

<div align="center">

Smart Bus Allocation & Transit Mapping System

Conductor Portal

Track your campus bus like a parcel — live, stop by stop.

Show Image Show Image Show Image Show Image Show Image

</div>

Table of Contents

[Overview](#)

[Features](#)

[System Architecture](#)

[Directory Structure](#)

[Prerequisites](#)

[Setup & Installation](#)

[API Reference](#)

[Usage Flow](#)

[Roadmap](#)

[Tech Stack](#)

[Author](#)

Overview

Ever refreshed a delivery app just to watch a little truck icon crawl across a map? That's the idea here — but for the campus bus.

The **Conductor Portal** gives a bus conductor a dead-simple interface to broadcast where their bus actually is, in real time: which stop it just left, how many seats are free, and where it's currently parked. Under the hood, a compiled **C engine** talks directly to SQLite with prepared statements, so even on modest hardware the whole pipeline — browser → Flask → C → database → back — stays fast enough to feel instant.

Features

 **Live seat counter** — conductor updates availability on the fly

 **Parking zone confirmation** — dynamic zone selection, no paper logs

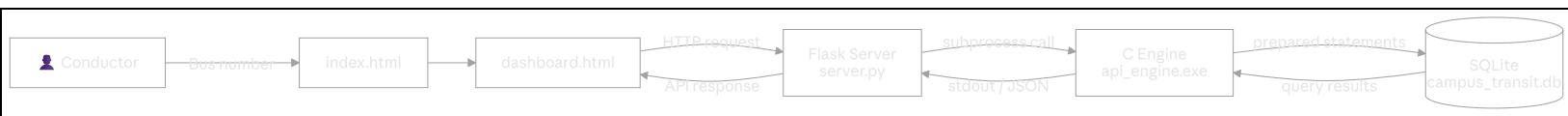
 **Sequential stop tracking** — "Arrived at Next Stop" advances the route like a parcel tracker

 **Shift-aware** — morning and evening shifts handled separately

- 🔥 **C-powered queries** — prepared statements straight to SQLite, no ORM overhead
- ✅ **Trip confirmation flow** — explicit confirm step before ending a trip, so nobody closes a route by accident

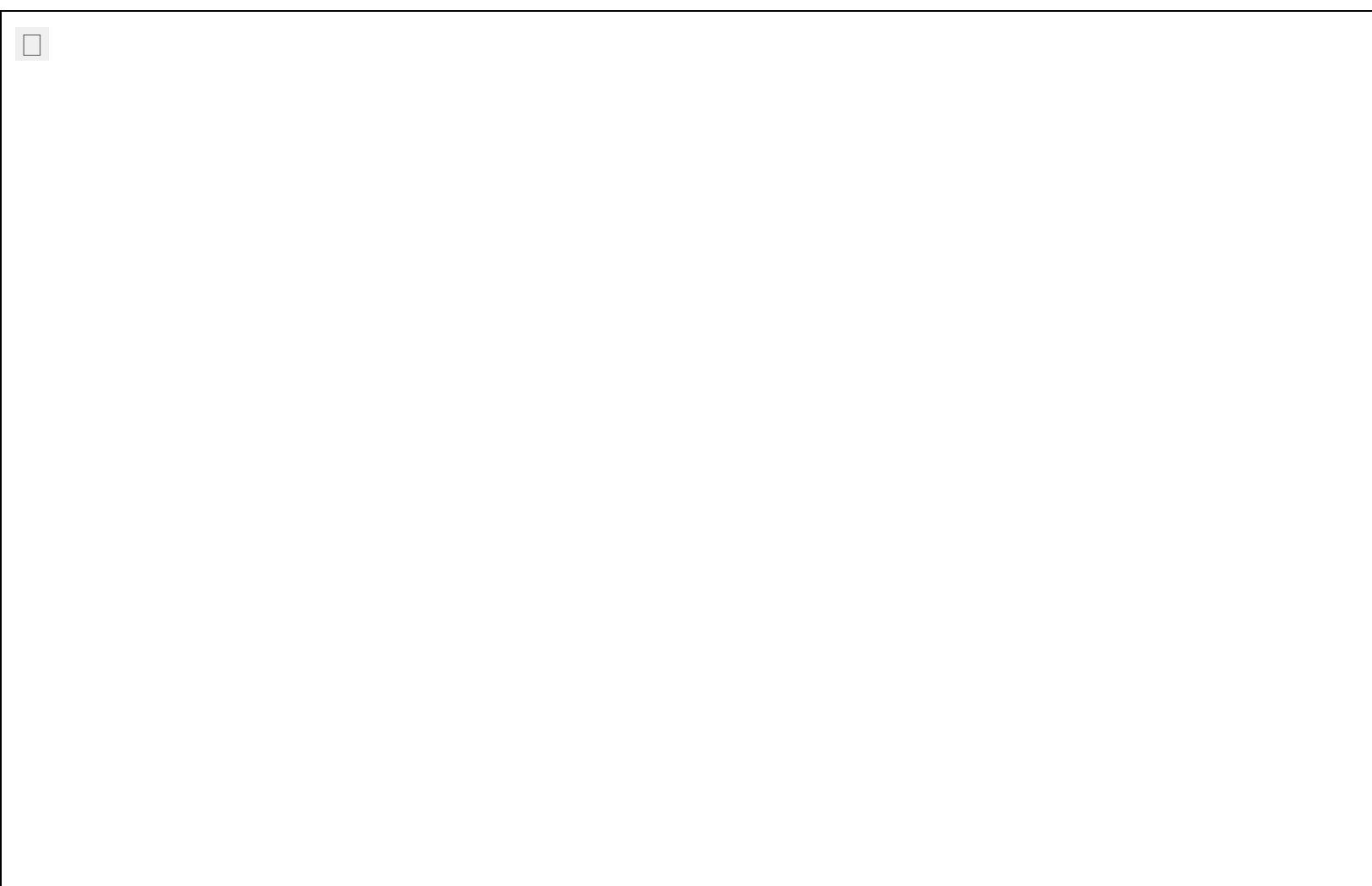
📁 System Architecture

Four layers, each doing exactly one job:



Layer	Responsibility
Frontend	Renders the portal, collects conductor input, polls for live state
API Bridge	Flask routes HTTP requests, handles CORS, shells out to the C binary
Core Engine	Compiled C binary — the only thing that touches SQLite directly
Database	Single source of truth for buses, schedules, and live status

📁 Directory Structure



```
├── C_engine/
│   ├── api_engine.c    # Core C backend handling SQLite queries
│   ├── sqlite3.c       # SQLite amalgamation source
│   └── sqlite3.h       # SQLite header
├── Data_Base/
│   ├── buses.csv       # Raw data source records
│   └── DataBase.sql     # Database initialization script
├── HTML/
│   ├── index.html      # Conductor login / bus-number entry portal
│   └── dashboard.html  # Main operations dashboard for seat/route tracking
├── JavaScript/
│   ├── api_engine.js   # API bridge adapter & data normalizer
│   ├── common.js       # Shared frontend utilities
│   └── data.js          # Static/fallback dataset definitions
├── Python/
│   └── server.py        # Flask API wrapper routing requests to the C engine
├── campus_transit.db    # Active SQLite database file
├── api_engine.exe       # Compiled C executable bridge
└── README.md
```

Prerequisites

Tool	Why you need it
Python 3.x + <code>flask</code> , <code>flask-cors</code>	Runs the API bridge
GCC (MinGW on Windows)	Compiles the C engine
SQLite3 dev headers	Lets the C engine link against SQLite

```

bash
pip install flask flask-cors
```

Setup & Installation

1. Compile the C Engine

From the project root:

```
bash
gcc C_engine/api_engine.c C_engine/sqlite3.c -o api_engine.exe -I C_engine
```

2. Start the Flask Backend

```
bash
python Python/server.py
```

The API bridge comes up at `http://localhost:5000`.

💡 **Tip:** if the C binary fails to launch from Flask, check that `api_engine.exe` sits where `server.py` expects it — a relative-path mismatch is the most common first-run snag.

3. Launch the Frontend

Open `HTML/index.html` with a local server (VS Code's Live Server extension works well) or directly in the browser.

🔌 API Reference

Endpoint	Method	Description
<code>/api/bus/<bus_id></code>	GET	Live bus details — registration, shift, conductor contact, current stop
<code>/api/bus/<bus_id>/parking-zone</code>	GET / POST	Read or confirm the bus's current parking zone

🔧 Seat-count updates, next-checkpoint advance, end-trip, and shift toggling are still wired to mock data on the frontend and are being connected to the live backend next.

📱 Usage Flow

Conductor Portal Entry (`index.html`) — enter your assigned bus number (e.g. `5`, `1`). The system checks it against SQLite.

Bus Dashboard (`dashboard.html`) — see registration number, shift, conductor contact, and the active route stop.

Live Operations — bump the seat count, advance to the next checkpoint, or confirm the current parking zone. Ending a trip requires confirmation, so nothing closes by accident.

Roadmap

- ☒ Conductor login via bus number
- ☒ Dashboard — seat counter, parking zone selector, sequential stop advance
- ☐ Wire the remaining API endpoints to live data (currently mock)
- ☐ **Student UI** — search bar + browse list, auto-refreshing every few seconds like a live parcel tracker
- ☐ Two-sided shift data — separate morning/evening trip records per bus
- ☐ Deployment build

Tech Stack

Layer	Technology
Frontend	HTML5 · Tailwind CSS · Vanilla JavaScript
API Bridge	Python · Flask
Core Engine C (prepared statements, no ORM)	
Database	SQLite

Author

Mukul — B.Tech Computer Science Engineering, Graphic Era University, Dehradun

<div align="center">

Built as a PBL project — because the campus bus deserves better than "should be here soon."

</div>